

EncodeBench

Measuring whether language models can write law as code, and what it takes to
measure that honestly

Max Ghenis*

2026-08-07

EncodeBench measures whether frontier language models can encode statutes into RuleSpec, the Axiom Foundation’s format for computable law: executable rules that cite their sources, carry their effective dates, and ship with tests. Sixteen UK provisions, resolved from a signed corpus release, grade on deterministic gates — the encoding compiles, passes the repository’s validation including its companion tests, and contains no number the source text lacks. Three boards to date cover six models. This paper reports the boards and, with equal weight, what running them taught about measurement: scores move with the toolchain that produced them, so a score is citable only against its encoder version; the harness gave different model families different exams before an audit caught it; and a reasoning-effort setting that appeared to control the comparison did nothing at all. The gates that grade the models also caught the harness. Board v4, pre-registered here, runs on an encoder that closes these gaps and reports means with spread in place of single draws.

Table of contents

1	Introduction	2
2	The task	3
3	Four gates and a fold	4
4	Three boards	5

*PolicyEngine and Axiom Foundation.

5	What it takes to measure this honestly	8
5.1	Scores bind to the toolchain	8
5.2	A timeout that graded slowness as failure	9
5.3	The effort setting that did nothing	9
5.4	Three backends, three exams	10
5.5	Case 04, flagged and retracted	10
5.6	Integrity notes	10
6	Board v4, pre-registered	11
7	Limitations	11
8	Reproducibility	12
	References	12

1 Introduction

The Axiom Foundation publishes statutes as executable code. An AI encoder drafts each encoding in RuleSpec — parameters and formulas that cite their statutory authority, carry their effective dates, and ship with companion tests — and a deterministic gauntlet decides what merges. That design turns model choice into an engineering question with money and correctness attached: which model drafts law-as-code well enough to sit inside a production pipeline, and how would you know?

EncodeBench makes the question measurable. It formalizes the informal bake-off the Axiom Foundation ran in July 2026 to route its own encoder, replacing a scratchpad comparison with a pinned suite, a deterministic fold, and public records. Code benchmarks grade models on tests (Chen et al. 2021; Jimenez et al. 2024) and legal benchmarks grade them on reasoning tasks (Guha et al. 2023); encoding law sits at the seam, where the output is code and the ground truth is a statute. Languages built for that ground truth (Mérigoux et al. 2021) and rules-as-code programs in government (Mohun and Roberts 2020) define the target; EncodeBench measures who can hit it. Its sister benchmark PolicyBench (Ghenis 2026) asks whether a model can compute the law; EncodeBench asks whether it can write it.

Three boards in, the results worth the most attention concern the measurement itself. Between two full-roster runs a day apart, five of six models moved, and the movement skewed positive — so a single board reads as one sample, not a ranking. A read-only audit found the three encoder backends administering different exams. The setting that appeared to pin reasoning effort did nothing on any backend. Each finding carries a disclosure against the boards it affects, each fix landed in the open, and the same deterministic machinery that grades the models caught every one of them. The paper reports the boards together with everything known to weaken them, then pre-registers the run that removes the weaknesses.

2 The task

One case gives one model one provision. The harness resolves the provision’s text from a signed corpus release — every run here pins uk-rulespec-2026-07-14, verified against its publishing key — and places it in a cold workspace: the source text, stubs for defined terms, and no existing encoding of the target. The model writes the RuleSpec module: the parameters and formulas the provision defines, citations back to it, effective dates, and the companion tests the repository requires of every rule. The suite covers UK law because the UK repository reached the signed-release toolchain the eval requires first; a US suite follows its release migration.

Capability cases run cold for a reason. A workspace with repository context includes whatever encoding of the provision already merged, so a capability score would come to measure copying, and would drift as the repository grows. Cold workspaces still carry definition stubs and canonical-concept context, identically for every model. The last three cases run with repository context as oracle candidates: they sit on the PolicyEngine UK comparison path and will carry a live oracle check once that plumbing is verified for eval cases; until then they grade on the same deterministic gates as everything else.

Table 1: The sixteen cases. Cases 1–13 run cold; cases 14–16 run with repository context as oracle candidates. SSCBA = Social Security Contributions and Benefits Act; ITEPA = Income Tax (Earnings and Pensions) Act; CB = Child Benefit (General).

#	case	provision	probes	family
01	vat_standard_rate	Value Added Tax Act 1994 s. 2	flat-rate control case	parameters
02	class_2_nic	SSCBA 1992 s. 13	weekly flat amount plus a small-profits threshold	parameters
03	class_1_secondary_nic	SSCBA 1992 s. 9	employer-side rate and secondary threshold	parameters
04	income_tax_rate_bands	Income Tax Act 2007 s. 10	basic, higher, and additional rate bands	bands
05	dividend_nil_rate	Income Tax Act 2007 s. 13A	band-conditional nil-rate amount	bands
06	personal_allowance_taper	Income Tax Act 2007 s. 35	£100,000 half-excess taper with round-up	tapers
07	hicbc_phaseout	ITEPA 2003 s. 681B	child benefit charge: £200 per £100 over the threshold	tapers
08	pension_annual_allowance	Finance Act 2004 s. 228	annual allowance beside the s. 228ZA taper	tapers
09	marriage_allowance_transfer	Income Tax Act 2007 s. 55B	transferable allowance: elections, both-party conditions	structure

#	case	provision	probes	family
10	income_tax_liability_steps	Income Tax Act 2007 s. 23	multi-step liability calculation;	structure
11	uc_award_calculation	Welfare Reform Act 2012 s. 8	import-heavy universal credit: maximum amount minus reductions	structure
12	child_benefit_entitlement	SSCBA 1992 s. 141	entitlement only — the rates live elsewhere	grounding
13	finance_act_2026_charge	Finance Act 2026 s. 1	recency probe: the 2026–27 income tax charge	grounding
14	income_tax_main_rates	Income Tax Act 2007 s. 6	main rates on the worker oracle path	oracle path
15	class_1_primary_nic	SSCBA 1992 s. 8	employee-side Class 1 contributions	oracle path
16	child_benefit_weekly_rates	CB Regulations 2006 reg. 2	the weekly rates case 12 must not invent	oracle path

Two cases exist to catch invented numbers. `child_benefit_entitlement` supplies entitlement conditions whose weekly amounts live in a different instrument — the 2006 regulations — so any rate literal in that encoding is an invention. `finance_act_2026_charge` encodes an act that postdates most of the roster’s training data, so the encoding must come from the supplied text rather than recall.

3 Four gates and a fold

A case passes for a model only when four deterministic checks pass: the encode returns an artifact, the artifact compiles in the Axiom rules engine, repository validation passes — including the companion tests — and the grounding scan finds zero numeric literals absent from the source text. The gate-pass count is the headline, and nothing else folds into it.

Everything else reports alongside. A pinned reviewer model scores each encoding’s statutory fidelity — a single, noisy judge, and one whose own family competes on the board, so it cannot referee neutrally and its score sits beside the gates rather than inside them. On board v3 its mean scores span 7.8 to 8.3 of 10 across all six runners — a band too narrow to separate the leaders, which is why it never gets to try. The oracle column stays empty until the PolicyEngine UK runtime path is verified for eval cases. The latency and cost columns carry measured values.

The roster carries a control: gpt-5.6-luna, which prior internal measurement found weak at exactly this task. Across the three boards it scored 5, 5, 8 of 16 — at or near the bottom every time. A board that failed to separate the control from the leaders would indict the instrument, not the models.

Runs fold into one board only when the suite name, the ordered case identities, the corpus release, and the normalized score-affecting toolchain identity all match. Runner sets may differ — a model added later folds cleanly without re-running incumbents — but a changed encoder refuses to fold: a new toolchain starts a new board by construction. Two of this paper’s findings turn on that refusal.

4 Three boards

Table 2: The three boards. Costs are API-equivalents: codex-backend tokens priced at published July 2026 rates, Claude-backend costs as the CLI’s own recorded per-call figures. All runs drew on subscription-billed seats.

board	run	encoder	runners	suite cost
v1	2026-07-23	0.2.1333	4	\$15.11
v2	2026-07-25	0.2.1380	6	\$40.25
v3	2026-07-26	0.2.1382	6	\$52.09

Board v1 piloted the suite on four codex-backend models. Boards v2 and v3 ran the full six-model roster one day apart on successive encoder versions — v2 on the encoder that carried the timeout defect of Section 5.2, v3 on the encoder that fixed it. v3 stands as the current board.

Table 3: Board v3 — encoder 0.2.1382, run 2026-07-26. Gate pass counts cases clearing all four gates. T counts cases that exhausted the recorded time budget; compile, ci, and grounded rates cover produced artifacts only. Reviewer and oracle columns are advisory and live in the board record.

runner	model	gate pass	T	artifacts	compile	ci	grounded	median
gpt-5.5	gpt-5.5	15/16 (93.8%)	—	16/16	100.0%	93.8%	100.0%	48s
sol	gpt-5.6-sol	14/16 (87.5%)	—	16/16	100.0%	87.5%	100.0%	45s
fable	claude-fable-5	12/16 (75.0%)	1	15/16	100.0%	80.0%	100.0%	266s

runner	model	gate pass	T	artifacts	compile	ci	grounded	median
terra	gpt-5.6- terra	11/16 (68.8%)	—	16/16	100.0%	68.8%	100.0%	34s
luna	gpt-5.6- luna	8/16 (50.0%)	—	15/16	93.3%	53.3%	100.0%	50s
opus-5	claude- opus-5	6/16 (37.5%)	—	16/16	87.5%	37.5%	100.0%	47s

Table 4: Board v3 per-case grid. P = gate pass, F = validation failure, T = timeout, E = no artifact.

case	gpt-5.5	sol	fable	terra	luna	opus-5
01 vat_standard_rate	P	P	P	P	P	F
02 class_2_nic	P	P	P	P	F	P
03 class_1_secondary_nic	P	P	P	P	E	F
04 income_tax_rate_bands	P	F	P	F	F	F
05 dividend_nil_rate	P	F	P	F	P	F
06 personal_allowance_taper	P	P	F	F	F	F
07 hicbc_phaseout	P	P	P	P	F	P
08 pension_annual_allowance	P	P	F	F	P	P
09 marriage_allowance_transfer	P	P	T	F	F	F
10 income_tax_liability_steps	P	P	P	P	F	F
11 uc_award_calculation	P	P	P	P	P	P
12 child_benefit_entitlement	P	P	P	P	P	P
13 finance_act_2026_charge	P	P	P	P	P	F
14 income_tax_main_rates	P	P	P	P	P	F
15 class_1_primary_nic	F	P	F	P	P	F
16 child_benefit_weekly_rates	P	P	P	P	F	P

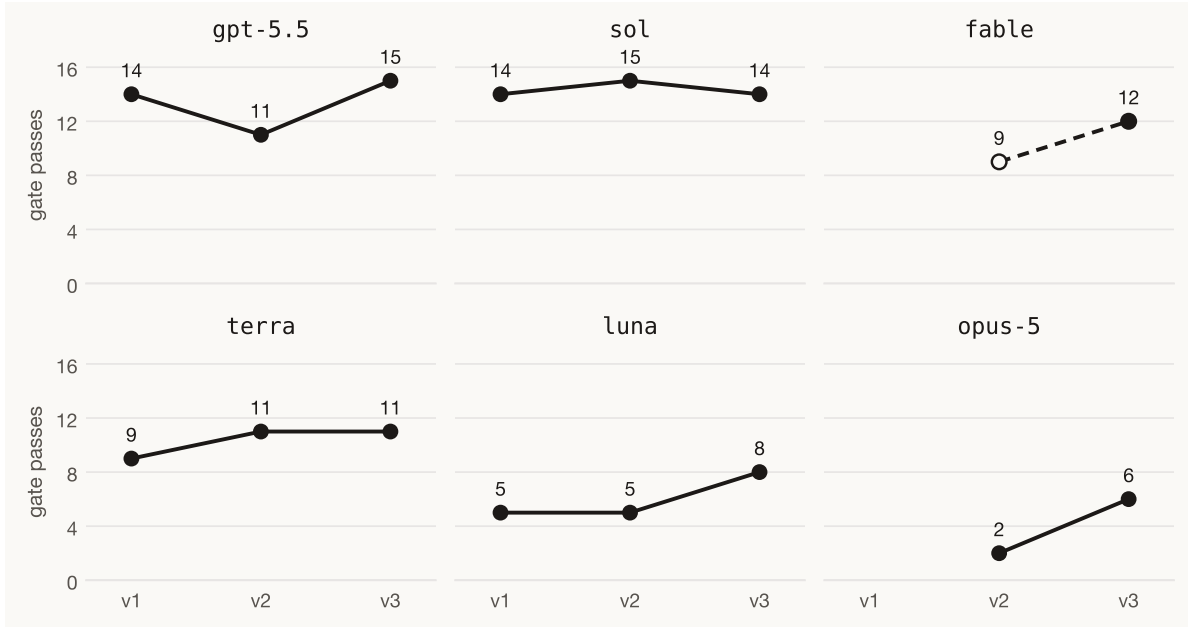


Figure 1: Gate passes per model across the three boards. Panels order by board v3 rank; the y-axis runs 0–16 cases. The boards ran on different encoder versions, so each line connects runs of the suite, not draws from one instrument. The hollow marker is board v2’s fable row, which a 600-second harness ceiling truncated (Section 5.2) — a floor, not a capability reading. fable and opus-5 joined the roster after v1.

Three readings of the current board survive the variance analysis that follows; the rest wait for board v4.

First, the extremes separate. gpt-5.5 and gpt-5.6-sol clear 15/16 and 14/16 while the control and claude-opus-5 sit at 8/16 and 6/16 — gaps far wider than anything the run-to-run movement produced.

Second, no model invented a number. Across the 250 artifacts produced over all three boards, the grounding scan found 0 with an ungrounded numeric — the two trap cases included. Frontier models do not fail this suite by fabricating rates; they fail it by writing encodings whose tests prove them wrong. The verification burden this implies sits in test quality, not fact-checking literals.

Third, speed and cost split by family, not by score. The codex runners’ median case ran 34–50 seconds on v3; fable’s ran 266 seconds and emitted 458,159 output tokens, putting its suite cost at \$33.02 against \$5.66 for sol — $5.8\times$ the cost for a lower score on this board.

Table 5: Board v3 cost annex, API-equivalent. The Claude CLI reports its own per-call cost, recorded verbatim; recomputing it from a rate table would mean inventing a rate for claude-opus-5, which has no published price in the reference the rig bundles. The Claude CLI also reports near-zero input tokens in this mode; its recorded cost already prices the full exchange.

runner	input tok	output tok	cache read	cost	per pass	basis
luna	826,676	43,734	310,528	\$1.12	\$0.14	rate table
terra	846,099	31,240	308,992	\$2.66	\$0.24	rate table
opus-5	36	152,826	774,613	\$3.85	\$0.64	CLI-reported
sol	897,438	33,026	356,608	\$5.66	\$0.40	rate table
gpt-5.5	898,057	37,576	344,576	\$5.79	\$0.39	rate table
fable	18	458,159	462,774	\$33.02	\$2.75	CLI-reported

5 What it takes to measure this honestly

5.1 Scores bind to the toolchain

Between board v2 (encoder 0.2.1380, 2026-07-25) and board v3 (0.2.1382, 2026-07-26), gate-pass moved gpt-5.5 +4, opus-5 +4, luna +3, fable +3, terra 0, sol -1 — five of six models moved, net +13. Movement that skewed positive rules out symmetric run-to-run noise as the whole story. Two causes confound, and two runs cannot separate them: model nondeterminism, and real toolchain change — the validator gained typed numeric occurrences and a locale profile between the boards, alongside the timeout fix. The fold’s refusal to combine boards across encoder versions stopped looking like bookkeeping here and started doing its job.

Four consequences govern how to read any single board:

- A board is one sample. gpt-5.5 leads v3 at 15/16 with sol at 14/16 — one case apart, from a model that moved four cases between boards. The order of ranks one and two is not a finding.
- The extremes are findings. The gaps separating the leaders from the control and from opus-5 exceed every delta the rerun produced.
- A score is citable only with its encoder version. This paper writes encoder versions next to every number for that reason, and the fold refuses to blend versions mechanically.
- The one ranking result that reproduced: terra below sol. terra scored 9, 11, 11 across the boards — the steadiest line any model produced — while sol scored 14, 15, 14. Two six-model boards on different toolchains and the four-model pilot agree. The July bake-off this benchmark formalizes had read the two as interchangeable on a smaller, easier case set; the finding that survives every board is that the larger, colder suite separates them.

5.2 A timeout that graded slowness as failure

Board v2’s harness carried a hardcoded 600-second ceiling on the Claude encoder path. The codex path exposed configurable timeouts; the Claude path hardcoded one constant that nobody had reason to read. fable’s median case ran 211 seconds on that board — roughly four to seven times the codex runners’ 29–57 — so the ceiling truncated exactly one model’s tail. The harness killed 4 of its sixteen cases with no artifact produced, and the fold scored each absent artifact as a compile and grounding miss. Among the 12 cases that completed, fable passed 9. The board record flagged the row as a floor the day it posted, and this paper does not cite it as a capability number anywhere.

The fix replaced the constant with a configurable, recorded budget — uniform 1,800 seconds per encoder invocation and 3,600 per case on v3 — and made timeouts visible: a case that exhausts its budget records `timed_out`, renders as T on the grid, and drops out of the artifact denominators instead of masquerading as a bad encoding. Execution identity now carries the timeout policy, so runs under different ceilings refuse to fold, the same way different encoders do.

Board v3 exercised the whole path on real artifacts. All six runners finalized with no operator salvage — on v2, two of six needed manual recovery after compile-failure text embedded ephemeral staging paths, a reproducibility defect the same fix normalized — and the board’s one timeout, fable’s `marriage_allowance_transfer` at 3,600 seconds of case budget, reads as exactly what it is.

5.3 The effort setting that did nothing

Every board above ran with a harness line that appeared to pin codex reasoning effort: `-c reasoning_effort="low"` on every codex invocation. `reasoning_effort` is not a recognized Codex configuration field — the real key is `model_reasoning_effort` — and codex invoked without `--strict-config` accepts unknown keys silently. The line never did anything. Every codex runner on every board ran at its receiver default, and the Claude path passes no effort flag at all, so those runners did too. A note posted to the tracking issue first reported the runs as pinned to low effort; receiver verification the same day corrected the record to inert. As a defect class this is worse than a wrong setting: the source reads as though the comparison is controlled.

A five-sample-per-level probe established what each knob does. On codex, the recognized key moves real resources: gpt-5.6-terra’s median reasoning tokens rose from 145 at low to 2756 at xhigh, median wall time from 20s to 53s, and ultra ran slowest (106s median) with the widest spread (401–6907 reasoning tokens). On the Claude CLI, `--effort` changed nothing measurable in print mode: cost, wall time, and answers held flat from low through max, and `claude-opus-5` returned the same answer at every level — consistent with the Claude 5 family regulating its own reasoning depth.

Cross-family effort matching therefore has no honest implementation on these receivers. The design that survives is asymmetric and says so: sweep the codex arms at declared, receiver-verified levels; run the Claude arms at their adaptive default and label them that way. Board v4 records requested and effective effort in execution identity, so effort-mismatched runs refuse to fold — the same mechanism that already refuses mixed toolchains.

5.4 Three backends, three exams

A read-only audit of the harness, posted in full to the tracking issue, found a deeper asymmetry: the three encoder paths did not administer the same exam. The codex path ran in a workspace and could shell-read the full context files. The Claude path ran prompt-only, with no tools. The OpenAI API path — not on the standing roster — truncated each context file to a 6,000-character prefix silently, and capped output at 16,384 tokens including reasoning, with partial artifacts eligible for scoring. CLI versions went unrecorded, and version drift changes which flags a CLI even accepts.

None of this shows on a leaderboard cell, and all of it shapes scores. The v1–v3 cross-family readings above therefore compare models on different evidence, and this paper flags them as weaker than the within-family readings wherever they appear.

The parity encoder closes the gap by construction rather than by calibration: every backend receives identical prompt bytes with all context inlined; the codex workspace starts empty; context overflow fails loudly as an infrastructure error instead of truncating silently; the harness rejects partial or truncated artifacts instead of scoring them; undeclared file access terminates the case; and CLI versions preflight and record per row.

5.5 Case 04, flagged and retracted

All 10 model-runs across boards v1 and v2 failed `income_tax_rate_bands`, and the v2 record flagged it as a suspected suite defect: at zero for ten, a broken fixture explained the data at least as well as a hard provision. Board v3 answered — gpt-5.5 and fable both passed it — and we retracted the flag on the tracking issue the day the board posted. Ten consecutive failures had measured difficulty, not validity. The operating rule the episode set: unanimous failure justifies reading the artifacts, never disqualifying the case from a summary table.

5.6 Integrity notes

The archived boards carry two integrity notes. opus-5’s first v2 run returned 1/16 with nine cases erroring in about a second each: its encoder and its own reviewer drew on one subscription account, and the account hit its session limit. We discarded that run as a measurement failure, split the accounts, and re-ran clean — zero quota errors, case durations of 10 to 465 seconds, and the executed model verified from traces; only the clean row ever published. luna’s v2

advisory-review column covered 8 of 16 after the shared reviewer account hit its own limit mid-run; the deterministic gates read only compile, tests, and grounding, so its gate column stood. Both events taught the same operating rule: an encoder never shares an account with its own judge.

6 Board v4, pre-registered

Board v4 runs when the parity encoder merges, and its design is fixed before any result exists:

- Same sixteen cases, same corpus release, new encoder — a new board by construction, folded by the same contract.
- One exam. Identical prompt bytes on every backend, context inlined, empty workspaces, overflow loud, partials rejected, undeclared access terminal, CLI versions recorded per row (Section 5.4).
- Effort as a recorded axis. Codex arms swept at declared, receiver-verified levels; Claude arms at their adaptive default, disclosed as such (Section 5.3). Effort-mismatched runs refuse to fold.
- Repeated runs per arm on the pinned encoder, reported as mean and spread. Single draws retire (Section 5.1).

v4 is the first board eligible to support a cross-family capability comparison. This paper refreezes with it: new snapshot, re-derived numbers, same standing rule that no number appears here that the frozen artifacts cannot reproduce.

7 Limitations

Sixteen cases sample one jurisdiction on one corpus release; the suite measures UK statutory encoding on this release, not encoding in general. The gates check that an encoding compiles, passes its companion tests, and grounds its numerics — they do not check full statutory fidelity, and the oracle column that would cross-check computed results against PolicyEngine UK stays empty until that runtime path is verified for eval cases. Each (model, board) cell is a single run, so within-toolchain variance stays unmeasured until v4’s repeated runs. Costs are API-equivalents priced from recorded tokens for the codex runners and CLI-reported figures for the Claude runners. The runners are CLI products, not bare models: each brings its own system prompt, tool behavior, and retry policy, and boards v1–v3 did not equalize them (Section 5.4).

8 Reproducibility

Every number in this paper derives in code from the frozen run artifacts — sixteen signed results.json files across the three boards, hashed file-by-file in the snapshot manifest — through a results module that re-implements the deterministic pieces of the eval-board fold. The module verifies its derived headlines against the board records posted to the tracking issue and refuses to render the manuscript on any mismatch; this render passed that check (verified against published records: 16 gate counts, medians, v3 artifacts/timeouts/costs).

[axiom-encode](#) holds the suite manifest, the harness, and the fold; the boards post to tracking issue [#1189](#), and [PR #1304](#) carries the parity encoder. The current board, the suite, and this paper live at [encodebench.org](#); the paper’s source, snapshot, and derivation module live in the [site repository](#) under paper/.

References

- Chen, Mark, Jerry Tworek, Heewoo Jun, et al. 2021. Evaluating Large Language Models Trained on Code. <https://arxiv.org/abs/2107.03374>.
- Ghenis, Max. 2026. PolicyBench: Benchmarking No-Tool Tax-and-Benefit Estimation in Frontier Language Models. <https://policybench.org/paper>.
- Guha, Neel, Julian Nyarko, Daniel E. Ho, Christopher Ré, Adam Chilton, et al. 2023. “LegalBench: A Collaboratively Built Benchmark for Measuring Legal Reasoning in Large Language Models.” Advances in Neural Information Processing Systems, Datasets and Benchmarks Track. <https://arxiv.org/abs/2308.11462>.
- Jimenez, Carlos E., John Yang, Alexander Wettig, et al. 2024. “SWE-Bench: Can Language Models Resolve Real-World GitHub Issues?” International Conference on Learning Representations. <https://arxiv.org/abs/2310.06770>.
- Mérigoux, Denis, Nicolas Chataing, and Jonathan Protzenko. 2021. “Catala: A Programming Language for the Law.” Proceedings of the ACM on Programming Languages 5 (ICFP). <https://arxiv.org/abs/2103.03198>.
- Mohun, James, and Alex Roberts. 2020. Cracking the Code: Rulemaking for Humans and Machines. OECD Working Papers on Public Governance No. 42. OECD Observatory of Public Sector Innovation.